Midterm Exam, Faculty of Engineering, Chulalongkorn University
Course ID: 2110215 Course Name: Programming Methodology I

Second Semester, Date: 11 March 2020 Time: 8.30-11.30AM

Name ..... Student ID. ..... No. in CR .....

Instructions for on-site students (online students - just ignore what you cannot do)

1. Write your Student ID, full name, and your number in CR58 in the space provided on this page.
2. Your answer must be on the computer center's machine in front of you.
3. Documents and files are allowed inside the exam room; however, internet and flash drive are prohibited. Borrowing is not allowed unless it is supervised by the proctor.
4. You must not carry mobile phone and flash drive during the exam. Online students must use mobile phone as a Zoom camera.
5. No communication is allowed.
6. ***** You must not bring any part of this exam paper outside. The exam paper is a government's property. Violators will be prosecuted under a criminal court and will receive an F in the subject. *****
7. Students who wish to leave the exam room before the end of the exam period, must raise their hands and ask for permission before leaving the room. Students must leave the room in the orderly manner.
8. Online students are not allowed to leave the online exam until the exam finishes.
9. Once the time expires, student must stop typing and must remain seated quietly until the proctors collect all the exam papers or given exam booklets. Only then, the students will be allowed to leave the room in an orderly manner.
10. Any student who does not obey the regulations listed above will receive punishment under the Faculty of Engineering Official Announcement on January 6, 2003 regarding the exam regulations.
    a. With implicit evidence or showing intention for cheating, student will receive an F in that subject and will receive an academic suspension for 1 semester.
    b. With explicit evidence for cheating, student will receive an F in that subject and will receive an academic suspension for 1 year.

Please sign and submit

Signature (.....)

Important Rules

- You must not bring any part of this exam paper outside. The exam paper is a government's property. **Violators will be prosecuted under a criminal court and will receive an F in the subject.**
- It is a student's responsibility to check the file. If it is corrupted or cannot be open, there is no score.
- Each question shows whether or not you have to modify or create each method or variable.

** Noted that Access Modifier Notations can be listed below*

+ (public)
# (protected)
- (private)
<u>Underline</u> (static)
Italic (abstract)

Set-Up Instruction

- Set workspace to "**C:\temp\progmeth2021_2\midterm_2110215_(your id)_(FirstName)**" (if not exist, you must create it)
  - For example, "C:\temp\progmeth2021_2\midterm_2110215_6470156021_Boss"
- All your files must be in the workspace.
- **The code outside of the workspace will not be collected, or graded**

To submit

<u>For on-site students:</u>

Just leave your IDE on. The TA will save the file from Computer Center for you.

<u>For online students</u>:

1.1. go to the workspace folder that you actually do the coding for the 3 questions of this exam.

1.2. Zip the workspace (there must be 3 folders, 1 for each question) to .zip and name it using your ID (for example, 6431300021.zip)

1.3. Submit the zipped file as an assignment on MyCourseville.

Question 3: (60 minutes, 10.30-11.30)

1. Instruction

1) Create Java Project named "**2110215_Midterm_Q3**".
2) Copy all folders in **"Q3"** to your project directory src folder.
3) You are to implement the following classes (detail for each class is given in section 3 and 4
   a) **BaseCard** (package logic.card)
   b) **RobotCard** (package logic.card)
   c) **MageCard** (package logic.card)
4) Make UML class diagram for classes in package logic.card, using any tool you choose. Save its picture file in your logic.card folder.
5) JUnit for testing is in package test.

2. Problem Statement: Card Duelist

(This question is based on the game Inscryption by Daniel Mullins Games, released in October 2021.)

We create a card game again called Card Duelist. This game is very similar to Card Fighter but it's more like the original game. This time there is no special attack effect in different cards, but different ways to summon it.

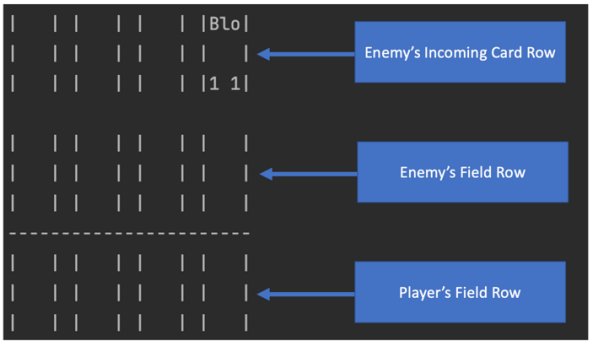
(This image of the game is not entirely accurate to the game of this question)

2.1 Game Component

1) Player

- At the start of the game, each player will have a damage point set to 0. If the player's damage point is equal to or more than opponent's damage point by 5 points, you win. You lose if the opposite happens.

2) Field



- Player can summon a unit card from their hand to a field, each player has 4 spaces to place his/her units (Player use lower row, opponent use middle and upper row) middle row is current enemy cards and upper row is incoming enemy cards row. You **can't** replace a card on the board with different card from hand if the card does not say so.

3) Deck

- Contains as many cards as you want, you can get more card by visiting some place.

4) Card

- All card has Name, Attack power, Health point.
- Player may play a unit card on to a field on any of 4 spaces, but the player has to follow the mechanic of the card.
- Unit card on the field will attack a unit in the same column once on player's turn, dealing damage equal to its attack power. If it overkills, it will deal the excess damage to the incoming enemy card. And if there is no unit on the same column, the unit will deal damage to opposite player.
- When a unit card takes damage from unit's attack or damage spell and health point reduces to 0, that unit card will be removed from the field.
- There are 3 different types of cards.

4.1) Orb Card

- A card used for the set-up before playing Mage card. It has no play condition.

4.2) Robot Card

- This card uses energy as its playing cost.

4.3) Mage Card

- This card can be played if there is an orb card with element type as this card.
- This card's attack power will be increase by 1 for each orb card with same element type.

2.2 Game Flow

- This game is rogue-like game. Player can pick a place to go. For example, to get more card, to get an upgrade for card, or encounter enemy.
- When player encounters the enemy a card game phase will commence.
- At the start of the game, the player shuffles his deck and draw 4 cards from deck. While opponent will place some card in incoming cards row.

1) Start with the player turn, the player draws a card.
2) The player can choose to play as many cards as he/she wants.
3) When the player finished, cards will attack from left to right.
4) Start the opponent's turn, the card in incoming cards row will slide down to the opponent's field. Then, execute attack from left to right.
5) Opponent may place card in incoming cards row.
6) Repeat until win condition is met by any player.

3. Implementation Detail

The class package is summarized below.

*** In the following class description, only details of IMPORTANT fields and methods are given. ***

3.1 Package logic.card /* You must implement some class in this package */

3.1.1 class BaseCard /* You must implement this class from scratch*/

This class is a base class for all kind of Cards. It contains all common elements which a card should have.

Field

Name	Description
- String name	Name of the card.
- int power	Power or attack point of the card.
- int health	Heath of the card.

Method

Name	Description
+ BaseCard(String name, int power, int health)	This is the Constructor. Set the parameter using proper setter
+ void play(Player player)	This method is called when the card is played in the **position** for the **player**. This method will perform action depending on each type of Cards.
+ boolean canPlay(Player player)	This method returns true if the card can be played according to the rule and returns false when the card is illegal to play. The rules are different for each type of Cards. No need to check if the field is full.
+ int attack(BaseCard target)	This method is called when this card attacks target. It will decrease target's health by its power but not lower than 0. And **return excess damage.** If the attack does not deal excess damage, return 0. E.g., "Power 5 card" attack "Health 2 Card" will return 3
+ void setPower(int power)	Setter for power. If power is less than 0, set it to 0
+ void setHealth(int health)	Setter for health. If health is less than 0, set it to 0
+ getter-setter for all fields.	

3.1.2. class OrbCard /*ALREADY PROVIDED*/

This class is a type of Card. This card does not have condition to be played. Use to summon mage card.

Field

Name	Description
- Element orbType	Element type of the card.

Method

Name	Description
+ OrbCard(String name, int power, int health, Element orbType)	This is the Constructor. Set the information of the super class. Set orbType as one given in the parameter.
+ void play(Player player)	This card does not have any action when played.
+ boolean canPlay(Player player)	This card does not have special conditions and can always be played. So, this will return true.
+ getter-setter for all fields.	

3.1.3. class RobotCard /* You must implement this class from scratch */

This class is a type of Card. This card's play condition is player must have enough energy and use energy according to its cost.

Field

Name	Description
- int energyCost	Energy cost of the card.

Method

Name	Description

+ RobotCard(String name, int power, int health, int energyCost)	This is the Constructor. Set the information of the super class. Set energyCost as one given in the parameter.
+ void play(Player player)	Subtract **player** energy by this card's **energy cost. Player's energy cannot be negative.**
+ boolean canPlay(Player player)	Return true if **player** have enough energy to play this card. Otherwise, return false.
+ void setEnergyCost(int energyCost)	Setter for Energy cost. If Energy cost is less than 0, set it to 0
+ getter-setter for all fields.	

3.1.4. class MageCard /* You must implement this class from scratch*/

This class is a type of Card. This card can only be played if there is orb card with same element type with this card. When play this card, this card power increase by 1 for each orb card with same element type on player's field.

Field

Name	Description
- Element mageType	Element type of the card.

Method

Name	Description
+ MageCard(String name, int power, int health, Element mageType)	This is the Constructor. Set the information of the super class. Set mageType as one given in the parameter.
+ void play(Player player)	Increase this card's **attack power** by number of orb card with the same element on player's field.

+ boolean canPlay(Player player)	Return true if **player** have orb card with the same element on player's field. Otherwise, return false.
+ getter-setter for all fields.	

3.2 Package logic.game

3.2.1. enum Element /*ALREADY PROVIDED*/

This enum contains element type for orbs and mages.

3.3 Package logic.player

3.3.1 class Player /*ALREADY PROVIDED*/

This class contains a method which will be used for storing player's data.

Field

Name	Description
- int damage	Damage that player have taken.
- int energy	Amount of energy that player has.
- ArrayList<BaseCard> deck	Player's deck
- ArrayList<BaseCard> hand	Player's hand
- ArrayList<BaseCard> field	Player's field

Method

Name	Description
+ Player(ArrayList<BaseCard> deck)	This is the Constructor. Set deck as one given in the parameter.
+ boolean playCard(int indexHand, int position)	This method played the card that in player's hand at **indexHand** to the selected **position** on player's field. If it can be played, remove that card from player's hand then return true, otherwise do nothing, and return false.

+ void drawCard()	This method draws first card from deck to player's hands.
+ getter-setter for all fields.	

3.4 Package application

3.4.1 class Main /*ALREADY PROVIDED*/

This class is the main program. You don't have to implement anything in this class. You can test the program by running this class.

3.5 Package test.student /*ALREADY PROVIDED*/

This package provides JUnit test for each of your class.

4. Score Criteria

The maximum score for the problem is 35 and will be rounded down to 5.

4.1 Implement Class BaseCard correctly = 12 points.

4.2 Class OrbCard: = 4 points

testConstructorLegalValue = 1 point

testConstructorIllegalValue = 1 point

testAttackNoExcess = 1 point

testAttackWithExcess = 1 point

4.3 Class RobotCard: = 8 points

testConstructorLegalValue = 1 point

testConstructorIllegalValue = 1 point

testSetEnergyCost = 1 point

testCanPlay = 1 point

testPlay = 1 point

testPlayNoEnergy = 1 point

testAttackNoExcess = 1 point

testAttackWithExcess = 1 point

4.4 Class MageCard: = 6 points

testConstructorLegalValue = 1 point

testConstructorIllegalValue = 1 point

testCanPlay = 1 point

testPlay = 1 point

testAttackNoExcess = 1 point

testAttackWithExcess = 1 point

4.5 UML = 5 points

All classes in package logic are present (correct variables and methods) = 2 points

Inheritance and Abstract structures are correct = 3 points